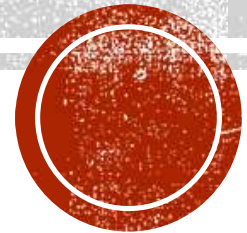


# **COS 201- COMPUTER PROGRAMMING ONE**

**MISS OYEWOLE CHARITY**



# JAVA DATA TYPES

Variables are nothing but reserved memory locations to store values. This means that when you create a variable you reserve some space in the memory.

Based on the data type of a variable, the operating system allocates memory and decides what can be stored in the reserved memory. Therefore, by assigning different data types to variables, you can store integers, decimals, or characters in these variables.

There are two data types available in Java

- Primitive Data Types
- Non-Primitive Data Types



# PRIMITIVE DATA TYPES

Java defines eight primitive types of data: **byte**, **short**, **int**, **long**, **char**, **float**, **double**, and **Boolean**.

The primitive types are also commonly referred to as simple types. These can be put in four groups:

- **Integers:** This group includes byte, short, int, and long, which are for whole-valued signed numbers.
- **Floating-point numbers:** This group includes float and double, which represent numbers with fractional precision.
- **Characters:** This group includes **char**, which represents symbols in a character set, like letters and numbers.
- **Boolean:** This group includes Boolean, which is a special type for representing true/false values.



# INTEGERS

- Java defines four integer types: byte, short, int, and long. All of these are signed, positive and negative values.
- The width and ranges of these integer types vary widely, as shown in this table:

| Name  | Width | Range                                                   |
|-------|-------|---------------------------------------------------------|
| long  | 64    | -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 |
| int   | 32    | -2,147,483,648 to 2,147,483,647                         |
| short | 16    | -32,768 to 32,767                                       |
| byte  | 8     | -128 to 127                                             |

- **BYTE:** The smallest integer type is byte. This is a signed 8-bit type that has a range from -128 to 127. Variables of type byte are especially useful working with a stream of data from a network or file. They are also useful when working with raw binary data that may not be directly compatible with Java's other built-in types.
- Byte variables are declared by use of the byte keyword.
- For example, the following declares two-byte variables called b and c: `byte b, c;`



# INTEGERS - SHORT, INT, LONG

- **SHORT:** short is a signed 16-bit type. It has a range from -32,768 to 32,767. It is probably the least used Java type. Short data type can also be used to save memory as byte data type. A short is 2 times smaller than an integer. Here are some examples of short variable declarations: short s; short t;
- **INT:** The most used integer type is int. It is a signed 32-bit type that has a range from -2,147,483,648 to 2,147,483,647. In addition to other uses, variables of type int are commonly employed to control loops and to index arrays. Integer is generally used as the default data type for integral values unless there is a concern about memory. Example: int a = 100000, int b = -200000
- **LONG:** long is a signed 64-bit type and is useful for those occasions where an int type is not large enough to hold the desired value. The range of a long is quite large. This makes it useful when big, whole numbers are needed.



# FLOATING-POINT TYPES

- Floating-point numbers, also known as real numbers, are used when evaluating expressions that require fractional precision.
- For example, calculations such as square root, or transcendental functions such as sine and cosine, result in a value whose precision requires a floating-point type.
- There are two kinds of floating-point types, float and double, which represent single and double-precision numbers, respectively. Their width and ranges are shown here:

| Name   | Width in Bits | Approximate Range                      |
|--------|---------------|----------------------------------------|
| double | 64            | $4.9\text{e-}324$ to $1.8\text{e+}308$ |
| float  | 32            | $1.4\text{e-}045$ to $3.4\text{e+}038$ |



# FLOATING-POINT TYPES

- **FLOAT:** The type float specifies a single-precision value that uses 32 bits of storage. Variables of type float are useful when you need a fractional component, but don't require a large degree of precision.

For example, float can be useful when representing dollars and cents. Default values is 0.0f. Example: float f1 = 234.5f

- **DOUBLE:** Double precision, as denoted by the double keyword, uses 64 bits to store a value. This data type is generally used as the default data type for decimal values, generally the default choice.

All transcendental math functions, such as `sin( )`, `cos( )`, and `sqrt( )`, return double values. When you need to maintain accuracy over many iterative calculations, or are manipulating large-valued numbers, double is the best choice.



# CHARACTER

- In Java, the data type used to store characters is char.
- char data type is a single 16-bit Unicode character
- Minimum value is '\u0000' (or 0)
- Maximum value is '\uffff' (or 65,535 inclusive)
- The range of a char is 0 to 65,536. There are no negative chars.
- Char data type is used to store any character
- Example: char letterA = 'A'



# BOOLEAN

- Java has a primitive type, called Boolean, for logical values. It can have only one of two possible values, true or false.
- Boolean is also the type required by the conditional expressions that govern the control statements such as if and for.
- Boolean data type represents one bit of information.
- There are only two possible values: true and false.
- This data type is used for simple flags that track true/false conditions.
- Default value is false
- Example: Boolean one = true



# NON-PRIMITIVE DATA TYPES

- These are derived from primitive types or created by the programmer. They can store multiple values or complex data. They reference memory addresses instead of holding the value directly.
- These are following non- primitive data types available in Java programming language: **Array, Class and Interface.**

**ARRAY:** An array is the collection of homogeneous (or similar types) data type. An array is an object that holds a fixed number of values of homogeneous or similar data-type.

- The length of an array is assigned when the array is created and after creation, its length is fixed.
- Example:- `int a[]=new int[6];`



# NON-PRIMITIVE DATA TYPES — CONT'D

- **CLASS:** Class: - A class is a “user defined data type” from which objects are created of class. In general, class declarations can include components, and it consists of data and methods in the form of a unit.
  - (a) **Modifiers:** - A class can be public or default access.
  - (b) **Class name:** - The name of class should begin with an initial capital letter.
  - (c) **Body:** - The class body is enclosed by braces {}.
- **INTERFACE:** An interface is basically a kind of class. So, an interface is a collection of “methods” without actual definitions and “variables”. Thus, it is the responsibility of the class that to define and implement the codes of these methods.



# VARIABLES

- The variable is the basic unit of storage in a Java program.
- A variable is defined by the combination of an identifier, a type, and an optional initializer.

## Declaring a Variable

- In Java, all variables must be declared before they can be used.

***type identifier [= value][, identifier [= value] ...] ;***

## Types of variables

- Instance Variable
- Static/Class Variable
- Local Variable
- Parameters



# LOCAL VARIABLES

- Local variables are declared in methods, constructors, or blocks.
- Local variables are created when the method, constructor or block is entered, and the variable will be destroyed once it exits the method, constructor or block.
- Access modifiers cannot be used for local variables.
- Local variables are visible only within the declared method, constructor or block.
- There is no default value for local variables so local variables should be declared, and an initial value should be assigned before the first use.



# INSTANCE VARIABLES

- Instance variables are declared in a class, but outside a method, constructor or any block.
- Instance variables are created when an object is created with the use of the key word 'new' and destroyed when the object is destroyed.
- Access modifiers can be given for instance variables.
- The instance variables are visible for all methods, constructors and block in the class. • Instance variables have default values.
- Instance variables can be accessed directly by calling the variable name inside the class.
- However, within static methods and different class ( when instance variables are given accessibility) that should be called using the fully qualified name `ObjectReference.VariableName`



# CLASS/STATIC VARIABLES

- Class variables also known as static variables are declared with the static keyword in a class, but outside a method, constructor or a block.
- There would only be one copy of each class variable per class, regardless of how many objects are created from it.
- Static variables are stored in static memory.
- Static variables are created when the program starts and destroyed when the program stops.
- Visibility is similar to instance variables.
- Default values are same as instance variables.
- Static variables can be accessed by calling with the class name  
ClassName.VariableName



# JAVA OBJECT ORIENTED PROGRAMMING CONCEPTS

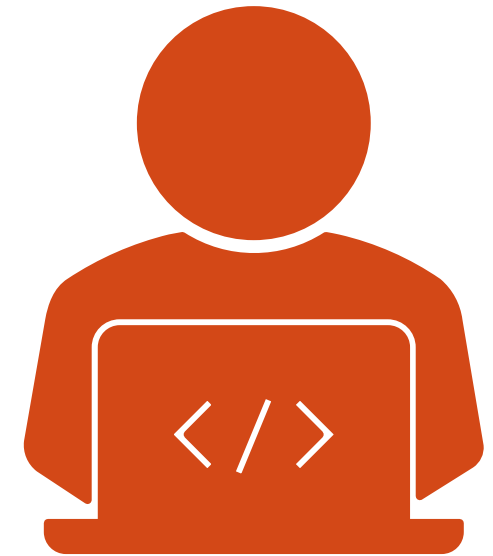
Java is an Object-Oriented Programming language which has a similar syntax to C/C++ programming languages but without its complexities. Java is widely used to develop mobile applications, desktop applications, web applications, web servers, games, and enterprise-level systems. Originally invented by James Gosling, Java is now owned by Oracle, with JDK 23 being its latest version. The language features a syntax similar to that of C and C++, making it familiar to many developers. Renowned platforms such as LinkedIn, Amazon, and Netflix utilize Java for their back-end systems, highlighting its reliability, scalability, and adaptability across various environments.



# OBJECT-ORIENTED PROGRAMMING CONCEPTS

Object-oriented programming (OOP) involves programming using objects. **Object-Oriented Programming** is a methodology to design a program using classes and objects. **An object** represents an entity in the real world that can be distinctly identified. **Object** means a real-world entity such as a pen, chair, table, computer, watch, etc. An object has a unique identity which include; state and behaviour.

- The state of an object (also known as its properties or attributes) is represented by data fields with their current values. A circle object, for example, has a data field radius, which is the property that characterizes a circle.
- The behaviour of an object (also known as its actions) is defined by methods. To invoke a method on an object is to ask the object to perform an action. For example, you may define a method named `getArea()` for circle objects. A circle object may invoke `getArea()` to return its area.



# BASIC PRINCIPLES OF OOP (OBJECT- ORIENTED PROGRAMMING)

- **Class** - A class is a template or blueprint from which objects are created. It describes a set of objects with the same behaviour. A class can contain Fields, methods, constructors, blocks, nested class and interface. To create a class, you use the keyword *class*.

```
Class class_name{  
    // variables declaration  
    // methods declarartion  
}
```

The variables declared in the class are known as instance variables.

The variables and methods declared within the curly braces are collectively known as members of the class.



# BASIC PRINCIPLES OF OOP (OBJECT- ORIENTED PROGRAMMING)

- **Object** - An object is an instance of a class. It is an entity that has state and behaviour. It can be physical or logical (tangible and intangible).

```
public class Main {  
    int x = 5;
```

```
public static void main(String[] args) {  
    Main myObj = new Main();  
    System.out.println(myObj.x);  
    }  
}
```

- **Abstraction** - The process of hiding the details and exposing only the essential features of a particular concept or object without including the background details or explanation.



# BASIC PRINCIPLES OF OOP (OBJECT- ORIENTED PROGRAMMING)

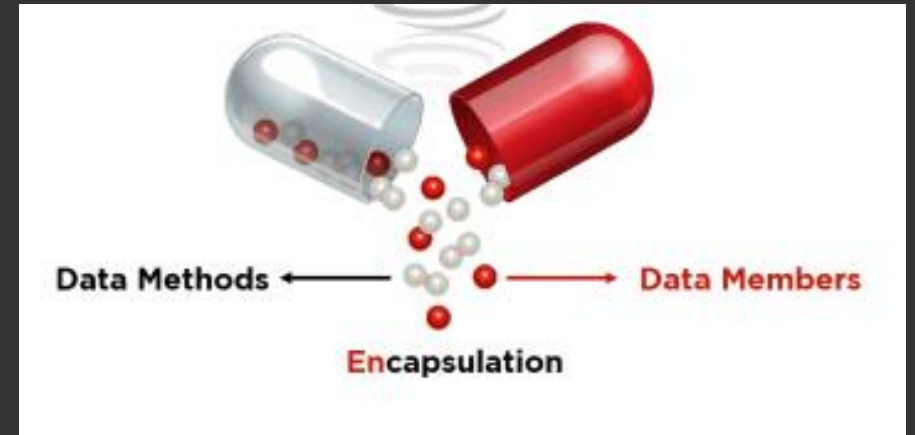
- **Inheritance** - Inherit members from parent class. Inheritance is the process of building new classes based on parent classes. The new class inherits the properties and attributes of the parent class
- **Polymorphism**- means the ability to take more than one form. An operation may exhibit different behaviour in different instances. The behaviour depends upon the types of data used in the operation. In java, we use **method overloading** and **method overriding** to achieve polymorphism.
- **Encapsulation** - Hide the internals of a class. This is wrapping up of data and functions into a single unit (called class).



# JAVA ENCAPSULATION

Encapsulation in Java refers to integrating data (variables) and code (methods) into a single unit. In encapsulation, a class's variables are hidden from other classes and can only be accessed by the methods of the class in which they are found.

Difference between **encapsulation** and **data hiding** is that Encapsulation is a way of bundling data whereas Data Hiding prevents external unauthorized access to the sensitive data. Access modifiers are used to achieve data hiding. Access modifiers are the keywords that specify the accessibility or the scope of methods, classes, fields, and other members



# TYPES OF ACCESS MODIFIERS IN JAVA

- **Public:** A public modifier is accessible to everyone. It can be accessed from within the class, outside the class, within the package, and outside the package.
- **Private:** A private modifier can not be accessed outside the class. It provides restricted access.
- **A protected modifier:** can be accessed from the classes of the same package and outside the package only through inheritance.
- **A default modifier** is accessible only within the package. If no modifier is used, it is treated as default.



# HOW TO IMPLEMENT ENCAPSULATION IN JAVA

There are two steps to achieve the purpose of Encapsulation in Java which are;

- Use the private access modifier to declare all variables/fields of class as private
- Define public getter and setter methods to read and modify/set the values of the above said fields

```
public class Person {  
    private String name; // private = restricted access  
  
    // Getter  
    public String getName() {  
        return name;  
    }  
  
    // Setter  
    public void setName(String newName) {  
        this.name = newName;  
    }  
}
```



# BENEFITS OF JAVA ENCAPSULATION

- Cleaner, more organized and less complex code.
- Makes the code more secure.
- The code can be maintained at any point without breaking the classes that use the code.
- Better control of class attributes and methods
- Flexible: the programmer can change one part of the code without affecting other parts
- Increased security of data



# TYPES OF JAVA PROGRAMS

Java programs can be developed in three different ways, which are briefly outlined here;

- **Java Applications:** These are stand-alone applications such word processors, inventory control systems etc.
- **Java Applets:** is a Java program that can be embedded into a web page. It runs inside the web browser and works at client side. These programs that are executed within a browser. They are executed on the client computer.
- **Java Servlets:** the Java programs that run on the Java-enabled web server or application server. They are used to handle the request obtained from the web server, process the request, produce the response, and then send a response back to the web server. These are server-side programs that are executed within a browser.





Package in Java is a mechanism to encapsulate a group of classes, sub packages and interfaces. Packages are used for:

- Preventing naming conflicts. For example, there can be two classes with name Employee in two packages, college.staff.cse.Employee and college.staff.ee.Employee
- Also define class members that are only exposed to other members of the same package
- Making searching/locating and usage of classes, interfaces, enumerations and annotations easier
- Providing controlled access: protected and default have package level access control. A protected member is accessible by classes in the same package and its subclasses. A default member (without any access specifier) is accessible by classes in the same package only.
- Packages can be considered as data encapsulation

# WHY JAVA?

- **Portability:** Java is a highly portable programming language because it is not designed for any specific hardware or software platform
- **Secure:** Java is a very secure programming language. Java codes (applets) may not access the memory on the local computer that they are downloaded upon. Thus, it provides a secure means of developing internet applications.
- **Simple:** Java's feature makes it a concise programming language that is easy to learn and understand.
- **Robustness:** Java is a strongly typed programming language and encourages the development of error free applications.



# PACKAGE

A Package can be defined as a grouping of related types(classes, interfaces). Java provides a large number of classes grouped into different packages based on their functionality.

The classes contained in the packages of other programs/applications can be reused. There are two types of packages in Java namely;

1. **Pre-defined Packages(built-in):** these packages consist of a large number of classes which are a part of Java API (Application Program Interface).
2. **User defined packages**

Pre-defined package include: java.lang, java.io, java.util, java.applet, java.awt, java.net, javax.swing and java.sql

- **Java.lang:** Contains language support classes (for e.g classes which defines primitive data types, math operations, etc.). This package is automatically imported.
- **Java.io:** Contains classes for supporting input / output operations.
- **Java.util:** Contains utility classes which implement data structures like Linked List, Hash Table, Dictionary, etc and support for Date / Time operations. This package is also called as **Collections**.

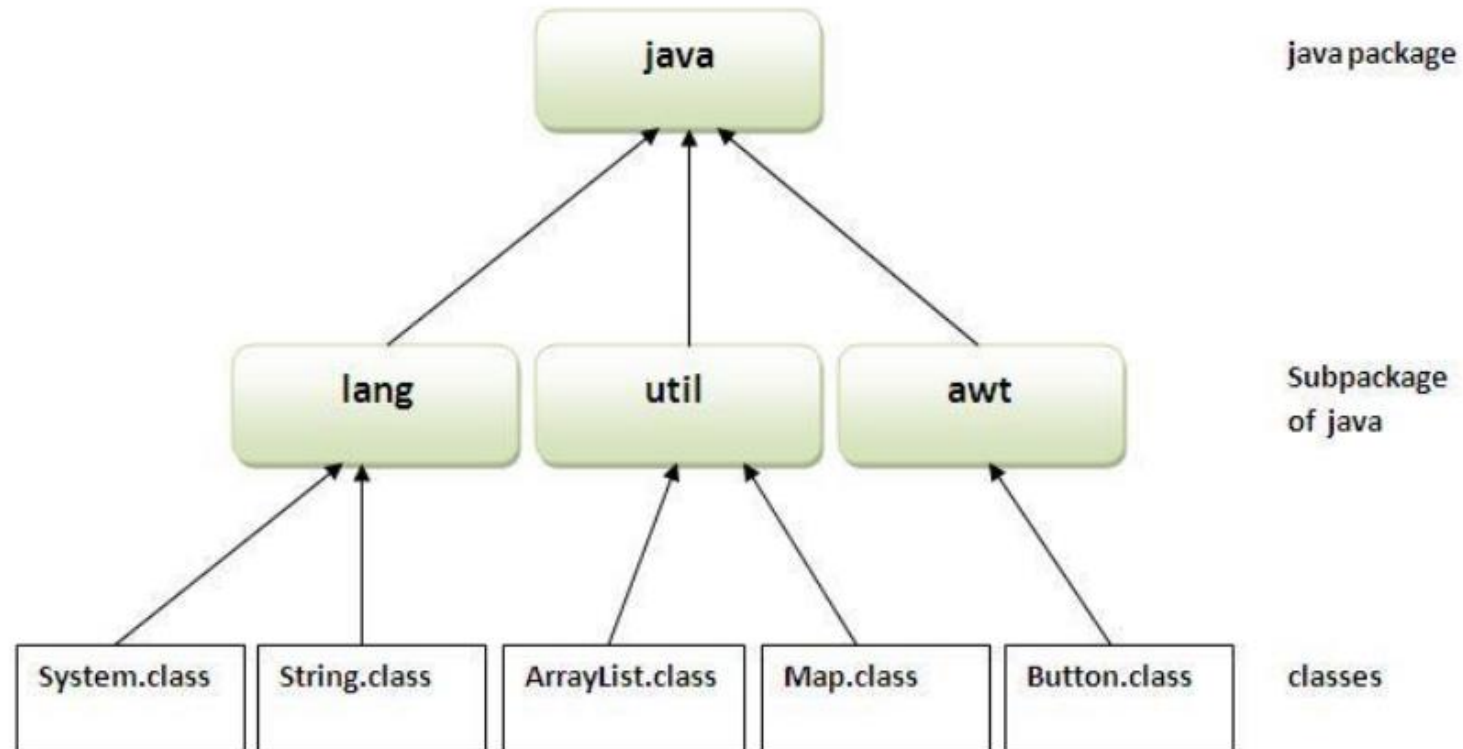


# PRE-DEFINED PACKAGE- CONT'D

- **Java.applet:** Contains classes for creating Applets.
- **Java.awt:** Contains classes for implementing the components of graphical user interface ( like buttons, menus, etc. ).
- **Java.net:** Contains classes for supporting networking operations.
- **Javax.swing:** This package helps to develop GUI Applications. The 'x' in javax represents that it is an extended package which means it is a package developed from another package by adding new features to it. In fact, javax.swing is an extended package of java.awt.
- **Java.sql:** This package helps to connect to databases like Oracle/Sybase/Microsoft Access to perform different operations.



# JAVA FOUNDATION PACKAGES



# HOW TO ACCESS PACKAGE FROM ANOTHER PACKAGE

To use a class or a package from the Java API, the keyword ***import*** should be used. The import keyword is used to make the classes and interface of another package accessible to the current package.

There are three ways to access the package from outside the package;

- Using packagename\*: To import a whole package, end the sentence with an asterisk sign (\*). The following example will import all the classes and interfaces in the *java.util* package but not subpackages:

```
import. java.util. *;
```



# ACCESSING CLASSES INSIDE A PACKAGE

- Import/calling a specific class by using packagename.classname: which means only declared class of this package will be accessible. E.g

*Import java.util.Vector;*

This imports only the vector class from the *java.util* package

- Using fully qualified name

If you use fully qualified name, then only declared class of this package will be accessible. Now there is no need to import. But you need to use fully qualified name every time when you are accessing the class or interface.

It is generally used when two packages have same class name e.g. *java.util* and *java.sql* packages contain Date class.



# USER-DEFINED PACKAGE

- These are packages defined by the user.
- User defined package is achieved by including a ***package*** command as the first statement in a Java source file. Any classes declared within that file will belong to the specified package.
- The package statement defines a name space in which classes are stored. If you omit the package statement, the class names are put into the default /package, which has no name.

General form of defining a package;

```
package pkg;
```

```
package major;
```



# HOW TO DEFINE A USER-DEFINED PACKAGE

```
package p1;  
    class c1  
    {  
        public void m1()  
        {  
            System.out.println("m1 of c1");  
        }  
        public static void main(string  
            args[])  
        {  
            c1 obj = new c1();  
            obj.m1();  
        }  
    }
```



# ASSIGNMENT

- Write out 10 classes in pre-defined package
- Write on Recursive algorithm



# EXCEPTION HANDLING

- **An exception (or exceptional event)** is a problem that arises during the execution of a program. When an Exception occurs the normal flow of the program is disrupted and the program/application terminates abnormally, which is not recommended, therefore, these exceptions are to be handled.
- The exception handling in java is one of the powerful mechanism to handle the runtime errors so that normal flow of the application can be maintained.
- Exception Handling is a mechanism to handle runtime errors such as ClassNotFoundException, IO, SQL, Remote etc.
- The core advantage of exception handling is to **maintain the normal flow of the application**. Exception normally disrupts the normal flow of the application that is why we use exception handling. Let's take a scenario:
  1. Code 1;
  2. Code 2;
  3. Code 3;
  4. Code 4; // exception occurs
  5. Code 5;
  6. Code 6;
  7. Code 7;



# EXCEPTION HANDLING

- The core advantage of exception handling is to **maintain the normal flow of the application**. Exception normally disrupts the normal flow of the application that is why we use exception handling. Let's take a scenario:

1. Code 1;
2. Code 2;
3. Code 3;
4. Code 4; // exception occurs
5. Code 5;
6. Code 6;
7. Code 7;

It is an unexpected problem that happens while your program is running. For example;

- Trying to divide a number by zero
- Trying to access a file that doesn't exist
- Trying to access an element in an array that doesn't exist.

When such problems occur, java creates an **exception object** that contains information about the error.



# TYPES OF EXCEPTION

There are mainly two types of exceptions which includes:

- Checked
- Unchecked exception
- Error; where Error is considered as unchecked expression
- **Checked Exception:** These are exceptions that are checked at compile-time. Examples include `IOException` and `SQLException`. They are problems that java forces you to handle before your program can run. The java compiler checks at compile time whether you've handled these exceptions. It can be handled by using **try-catch block** to handle the exception and by declaring the exception using the **throws** keyword in the method signature. Example such: Trying to read a file that doesn't exist (`FileNotFoundException`)
- **Unchecked Exception:** they are problems that occur due to logical errors in your code. The classes that extend `RuntimeException` are known as unchecked exceptions e.g. `ArithmeticException`, `NullPointerException`, `ArrayIndexOutOfBoundsException` etc. Unchecked exceptions are not checked at compile-time rather they are checked at runtime.



# TYPES OF EXCEPTION

- Examples include: dividing a number by zero (`ArithmeticException`), Accessing an index in an array that doesn't exist (`ArrayIndexOutOfBoundsException`). It can be handled using a **try-catch** block.
- **Error:** Error is irrecoverable i.e they are usually beyond your control and cannot be handled by your program. They often occur at the system level and are not meant to be caught or handled by your code. e.g. Running out of memory (`OutOfMemoryError`), `VirtualMachineError`, `AssertionError` etc



# SUMMARY OF EXCEPTION TYPES

| TYPE                | CHECKED/UNCHECKED | CAUSED BY                                | EXAMPLE                                   |
|---------------------|-------------------|------------------------------------------|-------------------------------------------|
| Checked Exception   | Checked           | External factors (files, databases, etc) | FileNotFoundException, SQLException       |
| Unchecked Exception | Unchecked         | Logical errors in code                   | NullPointerException, ArithmeticException |
| Errors              | Unchecked         | System-level failures                    | OutOfMemoryError, StackOverflowError      |



# COMMON SCENARIOS WHERE EXCEPTIONS MAY OCCUR

There are given some scenarios where unchecked exceptions can occur. They are as follows:

- **Scenario where `ArithmeticException` occurs**

If we divide any number by zero, there occurs an `ArithmeticException`.

```
int a=50/0;//ArithmeticException
```

- **Scenario where `NullPointerException` occurs**

If we have null value in any variable, performing any operation by the variable occurs an `NullPointerException`.

```
String s=null;
```

```
System.out.println(s.length());//NullPointerException
```



# COMMON SCENARIOS WHERE EXCEPTIONS MAY OCCUR

- **Scenario where `NumberFormatException` occurs**

The wrong formatting of any value, may occur `NumberFormatException`. Suppose I have a string variable that have characters, converting this variable into digit will occur `NumberFormatException`.

```
String s="abc";
```

```
int i=Integer.parseInt(s);//NumberFormatException
```

- **Scenario where `ArrayIndexOutOfBoundsException` occurs**

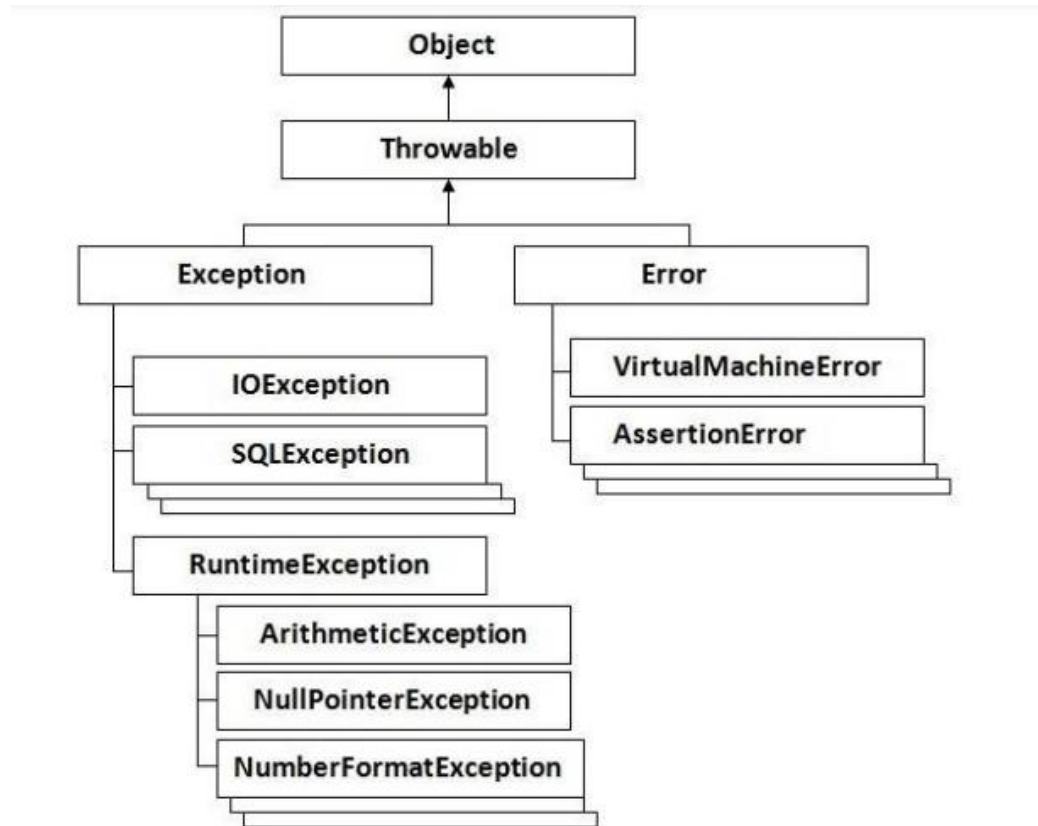
If you are inserting any value in the wrong index, it would result `ArrayIndexOutOfBoundsException` as shown below:

```
int a[]=new int[5];
```

```
a[10]=50; //ArrayIndexOutOfBoundsException
```



# HIERARCHY OF JAVA EXCEPTION CLASSES



# HIERARCHY OF JAVA EXCEPTION CLASSES

- In Java, exceptions are events that disrupt the normal flow of a program's instructions. The hierarchy of Java exception classes is rooted in the `Throwable` class, which is the superclass of all errors and exceptions in the Java language. Here's a detailed explanation of the hierarchy:
- **Throwable:** This is the root class for all exceptions and errors in Java. It provides a way to capture the state of the program when an exception or error occurs, including a stack trace and a message.
- **Exception:** This class and its subclasses represent conditions that a reasonable application might want to catch. Exceptions are divided into two main categories.
- **Checked Exceptions:** These are exceptions that are checked at compile-time. They must be either caught or declared to be thrown. Examples include `IOException` and `SQLException`.
- **IOException:** This is a general class of exceptions produced by failed or interrupted I/O operations.



# HIERARCHY OF JAVA EXCEPTION CLASSES

- **SQLException:** This provides information on a database access error or other errors related to SQL operations.
- **Unchecked Exceptions (Runtime Exceptions):** These are exceptions that are not checked at compile-time. They are usually the result of programming bugs, such as logic errors or improper use of an API. Examples include `ArithmeticException`, `NullPointerException`, and `NumberFormatException`.
- **ArithmeticException:** This is thrown when an exceptional arithmetic condition has occurred, such as division by zero.
- **NullPointerException:** This is thrown when an application attempts to use `null` in a case where an object is required.



# HIERARCHY OF JAVA EXCEPTION CLASSES

- **NumberFormatException:** This is thrown to indicate that the application has attempted to convert a string to one of the numeric types, but that the string does not have the appropriate format.
- **Error:** This class and its subclasses represent serious problems that a reasonable application should not try to catch. Errors are typically used to indicate issues with the Java Virtual Machine (JVM) itself or the environment in which the application is running. Examples include `VirtualMachineError` and `AssertionError`.
- **VirtualMachineError:** This indicates that the JVM is broken or has run out of resources necessary for it to continue operating.
- **AssertionError:** This is thrown to indicate that an assertion has failed.



# JAVA EXCEPTION HANDLING

There are 5 keywords used in java exception handling which are try, catch, finally, throw and throws.

## Java try-catch

Java try block is used to enclose the code that might throw an exception. It must be used within the method. Java try block must be followed by either catch or finally block

### Syntax of java try-catch

```
try{  
    //code that may throw exception  
}catch(Exception_class_Name ref){}
```

### Syntax of try-finally block

```
try{  
    //code that may throw exception  
}finally{}
```



# JAVA CATCH BLOCK

Java catch block is used to handle the Exception. It must be used after the try block only. You can use multiple catch block with a single try.

## **Problem without exception handling**

Let's try to understand the problem if we don't use try-catch block.

```
public class Testtrycatch1{  
    public static void main(String args[]){  
        int data=50/0;//may throw exception  
        System.out.println("rest of the code...");  
    }  
}
```

Output:

Exception in thread main java.lang.ArithmeticException:/ by zero



# SOLUTION BY EXCEPTION HANDLING

Let's see the solution of above problem by java try-catch block.

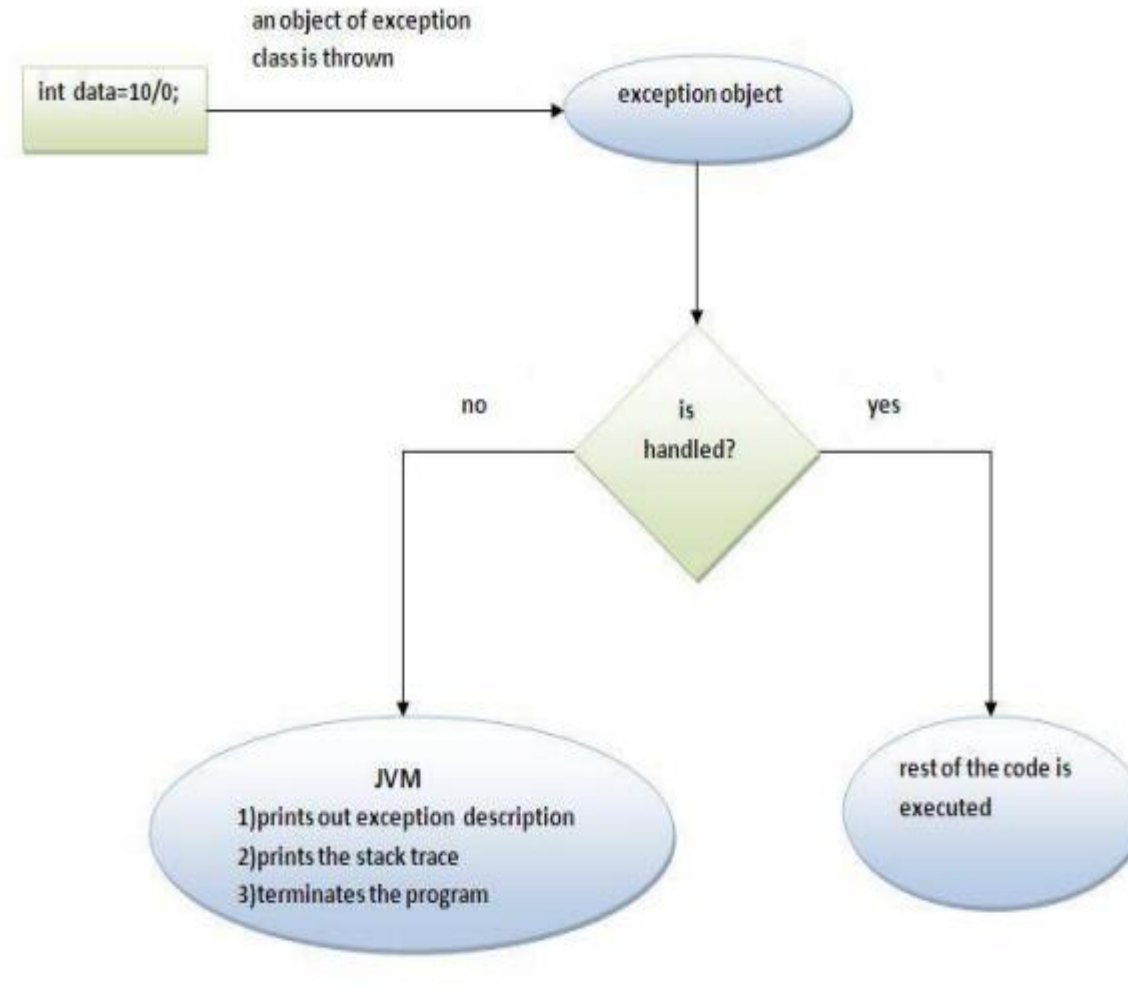
```
public class Testtrycatch2{  
    public static void main(String args[]){  
        try{  
            int data=50/0;  
        }catch(ArithmeticException e){System.out.println(e);}  
        System.out.println("rest of the code...");  
    }  
}
```

Output:

Exception in thread main java.lang.ArithmeticException:/ by zero  
rest of the code...



# INTERNAL WORKING OF JAVA TRY-CATCH BLOCK



# INTERNAL WORKING OF JAVA TRY-CATCH BLOCK

The Java Virtual Machine firstly checks whether the exception is handled or not. If exception is not handled, Java Virtual Machine provides a default exception handler that performs the following tasks:

- Prints out exception description.
- Prints the stack trace (Hierarchy of methods where the exception occurred).
- Causes the program to terminate.

But if exception is handled by the application programmer, normal flow of the application is maintained i.e. rest of the code is executed



# JAVA MULTI CATCH BLOCK

If you have to perform different tasks at the occurrence of different Exceptions, use java multi catch block.

Let's see a simple example of java multi-catch block.

```
public class TestMultipleCatchBlock{
    public static void main(String args[]){
        try{
            int a[]=new int[5];
            a[5]=30/0;
        }
        catch(ArithmeticException e){System.out.println("task1 is completed");}
        catch(ArrayIndexOutOfBoundsException e){System.out.println("task 2 completed");}
        catch(Exception e){System.out.println("common task completed");}
        System.out.println("rest of the code...");
    }
}
```

Output:task1 completed

rest of the code...



# JAVA MULTI CATCH BLOCK

- Rule: At a time only one Exception is occurred and at a time only one catch block is executed.
- Rule: All catch blocks must be ordered from most specific to most general i.e. catch for `ArithmeticException` must come before catch for `Exception`.

```
class TestMultipleCatchBlock1{  
    public static void main(String args[]){  
        try{  
            int a[]=new int[5];  
            a[5]=30/0;  
        }  
        catch(Exception e){System.out.println("common task completed");}  
        catch(ArithmeticException e){System.out.println("task1 is completed");}  
        catch(ArrayIndexOutOfBoundsException e){System.out.println("task 2 completed");}  
        System.out.println("rest of the code...");  
    }  
}
```

Output:

Compile-time error



# JAVA NESTED TRY BLOCK

- The try block within a try block is known as nested try block in java.

Why use nested try block?

- Sometimes a situation may arise where a part of a block may cause one error and the entire block itself may cause another error. In such cases, exception handlers have to be nested.



# JAVA NESTED TRY BLOCK

## Syntax

```
....  
try  
{  
    statement 1;  
    statement 2;  
    try  
    {  
        statement 1;  
        statement 2;  
    }  
    catch(Exception e)  
    {  
    }  
}  
catch(Exception e)  
{  
}  
.....
```



# JAVA NESTED TRY EXAMPLE

```
class Excep6{
    public static void main(String args[]){
        try{
            try{
                System.out.println("going to divide");
                int b =39/0;
            }catch(ArithmeticException e){System.out.println(e);}
            try{
                int a[]=new int[5];
                a[5]=4;
            }catch(ArrayIndexOutOfBoundsException e){System.out.println(e);}
            System.out.println("other statement");
        }catch(Exception e){System.out.println("handeled");}
        System.out.println("normal flow..");
    }
}
```

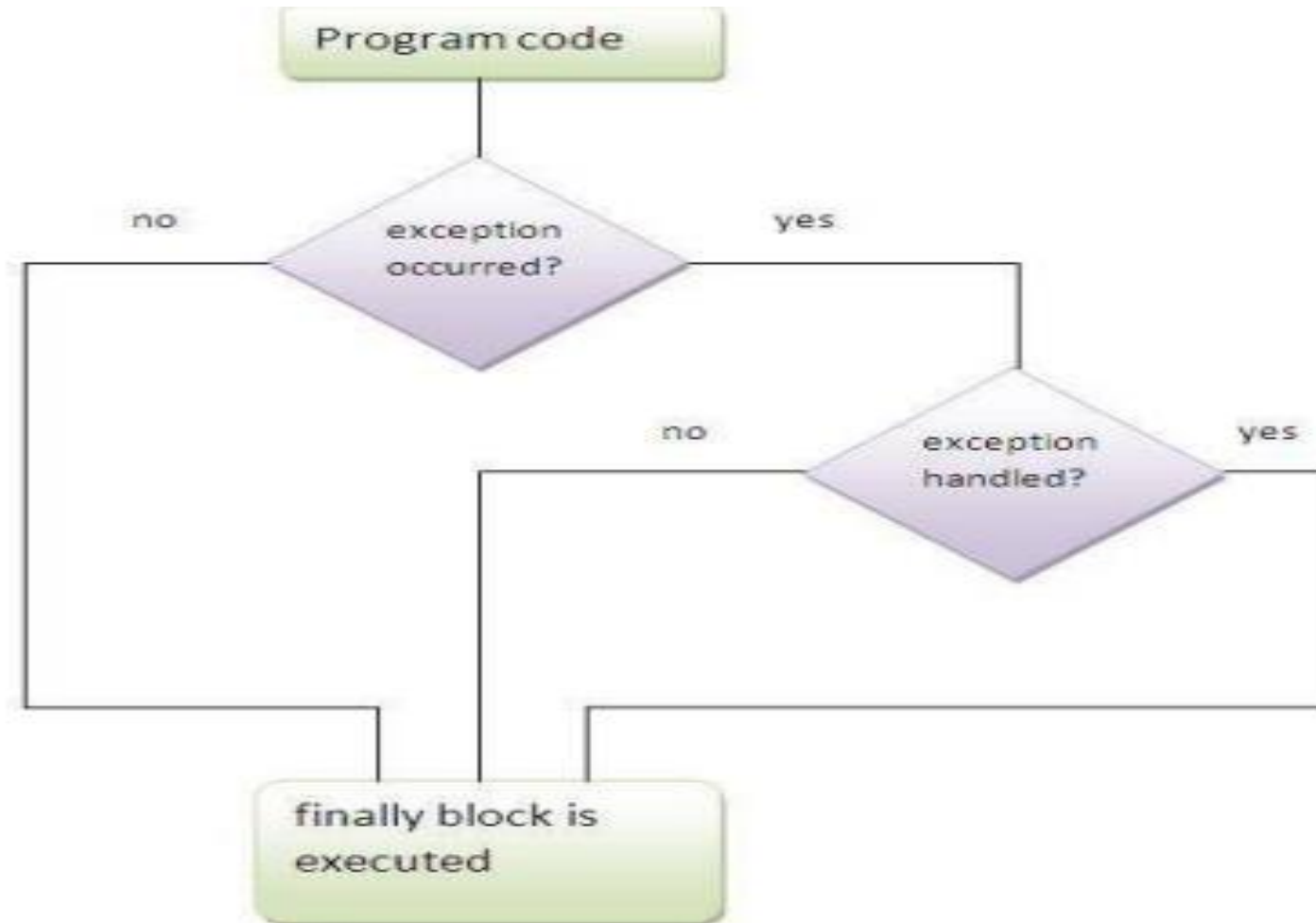


# JAVA FINALLY BLOCK

- Java finally block is a block that is used to execute important code such as closing connection, stream etc.
- Java finally block is always executed whether exception is handled or not.
- Java finally block follows try or catch block



# JAVA FINALLY BLOCK



# USAGE OF JAVA FINALLY

- Let's see the different cases where java finally block can be used.

## Case 1

**Let's see the java finally example where exception doesn't occur**

```
class TestFinallyBlock{  
    public static void main(String args[]){  
        try{  
            int data=25/5;  
            System.out.println(data);  
        }  
        catch(NullPointerException e){System.out.println(e);}  
        finally{System.out.println("finally block is always executed");}  
        System.out.println("rest of the code...");  
    }  
}
```

Output:5

finally block is always executed

rest of the code...



# USAGE OF JAVA FINALLY

Case 2 Let's see the java finally example where exception occurs and not handled.

```
class TestFinallyBlock1{  
    public static void main(String args[]){  
        try{  
            int data=25/0;  
            System.out.println(data);  
        }  
        catch(NullPointerException e){System.out.println(e);}  
        finally{System.out.println("finally block is always executed");}  
        System.out.println("rest of the code...");  
    }  
}
```

Output: Exception in thread main java.lang.ArithmeticException:/ by zero  
finally block is always executed  
rest of the code...



- Rule: For each try block there can be zero or more catch blocks, but only one finally block.
- Note: The finally block will not be executed if program exits (either by calling `System.exit()` or by causing a fatal error that causes the process to abort).

